

Smart Attendance — AI/ML Face Recognition Service

Project: Smart Attendance / Check-in System (Face Recognition)

Component owner role: AI/ML face-recognition service

Document: Design discussion & decision record

Date: 21 June 2026 | **Target demo:** Saturday, 27 June 2026 (group interview)

1. Project Overview

A smart attendance system that marks attendance using **live face recognition**. Built by a 5-member team, each owning one component, integrated via shared API contracts and GitHub feature branches.

Component	Responsibility
Flutter (mobile)	Camera captures face, sends image to backend
Spring Boot (backend)	Receives image, calls AI service, stores attendance in DB
AI/ML service (THIS)	Face recognition: image -> identity (local, pretrained, fast)
React (admin)	Dashboard: attendance logs, who checked in and when
Lead / DB / DevOps	Schema, integration, deployment glue

2. My Role and Hard Constraints

I own the **AI/ML face-recognition service** only. It talks to **Spring Boot only** (Flutter and React never call it directly).

Hard constraints agreed:

- **No external/cloud API** — no AWS Rekognition, Azure Face, Google Vision. Everything runs locally.
- **Pretrained model** — download weights once, zero training from scratch.
- **Fast / no delay** — recognition must feel instant.
- **Minimal storage, small hardware** — must fit a small edge device.

3. The Core Concept: Embeddings (not fingerprints)

The pretrained model is **not** trained to recognise specific people. It converts **any** face into a numeric vector called an **embedding** (512 numbers) — the same person's face always lands close together; different people land far apart.

Note: 'fingerprint' was only used as an **analogy** for this embedding. The project is **purely face recognition** — no finger/thumb biometrics anywhere.

Adding a person = enrollment = one-shot: run their face through the model once, store the embedding. No retraining. Recognising = embed the new face, compare to stored embeddings, pick the closest if it's within a threshold.

4. Architecture Decisions

Decision	Choice	Why
----------	--------	-----

Model / library	InsightFace buffalo_s (ArcFace, MobileFaceNet, Blazeface, etc.)	FastAPI + uvicorn ONLY ~10-30 ms/face; edge-friendly; pretrained
Runtime	onnxruntime	Runs model locally on CPU, no GPU, no API
Service framework	FastAPI + uvicorn	Local HTTP 'door' so Spring Boot can reach the model
State	Stateless service; backend DB owns embeddings	Database is source of truth; service is pure compute
Speed trick	Embeddings cached in RAM for matching	Matching is microseconds; ~2 KB per face (512 floats)
Matching owner	The AI service does the matching	Keeps ML logic + threshold on the AI side; backend stays simple

5. Does FastAPI break the 'no API' rule? No.

'API' is an overloaded word:

- **Avoided:** calling someone else's **cloud** service over the internet (costs money, needs WiFi, sends data out).
- **Used:** FastAPI is a **local doorway** on your own machine so two programs can talk. Nothing leaves the device.

FastAPI does not do recognition — the model does. FastAPI is just the messenger that lets Spring Boot hand over an image and get an answer back. It works fully offline (WiFi off).

6. How the Model Talks to Spring Boot

- Flutter -> photo -> Spring Boot
- Spring Boot -> **POST http://localhost:8000/recognize** (with the image) -> AI service
- AI service -> model embeds + matches -> returns {person_id, confidence}
- Spring Boot -> saves attendance record (who + timestamp) to its DB
- React -> reads records -> shows the dashboard

7. API Contract (the one coordination point with backend)

Endpoint	Input	Output
POST /enroll	{person_id, image}	{status, embedding}
POST /recognize	{image}	{person_id, confidence} 'unknown' 'no_face'
GET /health	-	{status: 'ok'}

8. Should We Fine-Tune the Model? No.

Fine-tuning is the wrong instinct here because face recognition uses **metric learning** — recognising a new person is a one-shot **enrollment**, not a training problem.

- It breaks the constraints (needs GPU, hours, a dataset; we want it fast and today).
- We don't have enough per-person data; small-data fine-tuning overfits and can make the model worse.
- Re-enrollment is free: add/remove a person = add/remove an embedding.

When fine-tuning IS legitimate: domain shift (thermal/IR cameras, heavy masks, very low-res CCTV), or when you are building the embedding model itself. Not our case (normal RGB webcam).

9. My Real Engineering Work (not 'copy-paste')

The pretrained model is ~10% of the system. The engineering is everything around it:

- **Threshold tuning** — choosing the similarity cut-off by testing false-accept vs false-reject.
- **Architecture** — stateless service + in-RAM matching + backend as source of truth.
- **API contract design** — the Python<->Java integration surface.
- **Edge-case hardening** — no face, multiple faces, unknown person, blurry image, persistence on restart.

10. How My Service Works — Step by Step

A. Startup (once)

- Load pretrained model into memory.
- Load enrolled embeddings from backend / local file into RAM.
- Listen on localhost:8000. No internet used.

B. Enroll a person

- Receive POST /enroll {person_id, image}.
- Detect face -> align -> embed (512 numbers).
- Store embedding in RAM and return it so backend saves it in DB.

C. Recognise (live check-in — hot path)

- Receive POST /recognize {image}.
- Detect face. If none -> 'no_face'. If multiple -> pick largest / reject.
- Align + embed the face.
- Cosine-compare vs in-RAM embeddings -> closest match + score.
- Above threshold -> {person_id, confidence}; below -> 'unknown'.
- Compare step takes microseconds = the 'no delay'.

11. How the Design Honours Every Constraint

Constraint	How it is met
Local / no cloud API	Model runs on device; zero internet; works WiFi-off
Fast / no delay	Embeddings in RAM; matching in microseconds; embed ~10-30 ms
Minimal storage	~2 KB per face; store embeddings, not photos
Small hardware	Tiny model (~tens of MB), CPU-only, runs on a Pi
Pretrained, no fine-tuning	Enrollment is one-shot embedding — correct by design

12. Demo Strategy (Saturday, group interview)

- Build the AI service to **stand alone** — demo works even if backend/Flutter aren't ready.
- Show: enrolled set -> live check-in -> instant match with name + confidence.

- **Killer move:** turn WiFi OFF and recognise a face — proves 'no API / fully local'.
- **Edge-case demo:** point at no one ('no_face'), then an un-enrolled person ('unknown') — proves it's a system, not a toy.
- Have a recorded video backup in case hardware misbehaves.

13. Repository & Team Workflow

GitHub repo: **NexTechANKARA/smart-attendance** (public). Branch per component:

- main — always working; no direct commits.
- feature/ai-face-service — my branch.
- feature/flutter-camera, feature/springboot-api, feature/react-dashboard.
- Integrate via Pull Requests into main; pull often to stay in sync.

14. Interview Soundbite

"I built a stateless recognition service around a pretrained embedding model. My work was the engineering around it: I designed the enroll/recognise API contract with the backend, implemented in-RAM matching for sub-millisecond lookup, tuned the similarity threshold against false-accept/reject rates, and handled the real-world edge cases — no face, multiple faces, unknown person, and persistence across restarts. The pretrained model is one dependency; the system is mine."